

AI Healthcare Operations & Administration Agent

Complete Project Documentation — Agent Architecture, Tools, Workflows, Appointment Management, FastAPI Backend, RBAC and Build Plan

Scope: A portfolio-grade administrative healthcare agent. It manages appointments, schedules, billing administration, insurance verification, analytics, policies, reports, communication and escalation. It is not a medical diagnosis or treatment recommendation system.

Development strategy: Build and test the complete agent first. Then build FastAPI/PostgreSQL, authentication and RBAC around the proven agent.

1. Project Overview

A user communicates naturally with the system. LangGraph maintains state and orchestrates a reasoning loop. LangChain exposes specialized tools. The LLM chooses capabilities and interprets results, while deterministic services perform actual database operations, calculations and external actions.

The key separation is: **the LLM decides what it needs; the application decides whether it is allowed; deterministic tools perform the operation.**

Final architecture

```
CLIENT | v FastAPI -> JWT -> User Identity / Role | v LangGraph Agent / Supervisor | +--> Planner / Reasoner +-->
Tool Selection +--> Permission Engine +--> Tool Execution |-- PostgreSQL services |-- Calendar / Scheduling |--
Billing / Insurance |-- Policy RAG |-- Web Search |-- Analytics |-- Communication |-- Reports |-- Escalation | v
Tool Results -> Agent Reasoning -> More Tools? -> Response | v Audit / Observability
```

2. Technology Stack

Layer	Technology	Purpose
API	FastAPI	REST API, validation, dependency injection
Agent orchestration	LangGraph	State, routing, conditional workflows, approvals
LLM framework	LangChain	Typed tools, prompts, model integration
LLM	Llama/Qwen via Groq	Fast inference
Database	PostgreSQL	Operational persistence
RAG	pgvector or Qdrant	Administrative policy retrieval
Cache/state	Redis	Caching, short-lived state and background work
Auth	JWT	Identity and authenticated access
Deployment	Docker/Compose	Reproducible runtime
Testing	pytest	Unit, agent, RBAC and API testing

External calendar, insurance, communication and search providers can initially be mock services. Their interfaces should remain clean so real providers can be substituted later.

3. Stage A — Build the Complete Agent First

Use synthetic patients, doctors, schedules, appointments, billing and insurance records. Implement every tool independently, then connect the tools to LangChain and LangGraph. Do not initially depend on FastAPI or production RBAC.

```
User request | v LangGraph state | v Planner / reasoning | v Tool selection | v Execute tool | v Observe result |
+---- more work? ----> next tool | v Final response
```

Stage-A acceptance criteria: correct tool selection, reliable multi-tool workflows, stateful booking, rescheduling/cancellation, policy retrieval, reporting, failure handling and no fabricated success.

4. Stage B — Secure Backend

After the agent is stable, build FastAPI and PostgreSQL around it. Add JWT authentication, roles, scope-aware permissions, approvals, audit logs and production hardening. The agent flow remains the same; the backend adds identity and security.

5. Complete Tool Catalog

Each tool should have a strict input schema, predictable output, explicit failure states and a defined side-effect level.

Category	Tool	Purpose	Example
Patient	patient_search	Find permitted records	Find P1004
Patient	patient_profile	Read permitted admin profile	Own profile
Patient	patient_history	Read administrative history	Past appointments
Scheduling	doctor_search	Find doctors	Find Dr. Ahmed
Scheduling	doctor_schedule	Get working schedule	Tuesday hours
Scheduling	appointment_search	Find authorized appointments	Tomorrow
Scheduling	appointment_create	Create confirmed appointment	Book 11:30
Scheduling	appointment_cancel	Cancel authorized appointment	Cancel booking
Scheduling	appointment_reschedule	Move appointment	Move Wednesday
Finance	billing_lookup	Invoice/payment status	Overdue invoices
Finance	insurance_verify	Verify insurance status	Active coverage
Intelligence	analytics	Deterministic metrics	No-show rate
Intelligence	policy_rag	Retrieve admin policy	Cancellation rule
Intelligence	web_search	Public/external information	Public clinic info
Operations	communication	Prepare/send messages	Reminder
Operations	report_generator	Generate report/PDF	Monthly report
Operations	escalation	Create staff ticket	Escalate
Operations	audit_log	Record action	Trace change

6. How Tool Calling Works

LLM requests: { "tool": "appointment_search", "arguments": { "patient_id": "P1004", "date": "tomorrow" } }
Pipeline: LLM call -> schema validation -> authorization -> service/tool -> normalized result -> LangGraph state
-> next decision

The model never executes arbitrary Python or unrestricted SQL. Registered tools are the only capabilities it can request. Deterministic calculations belong in code, not model reasoning.

7. Appointment Booking Workflow

Booking is stateful because users may provide doctor, date and time information across multiple turns.

USER: "I want to book an appointment with Dr. Ahmed." | v Intent = BOOK_APPOINTMENT | v doctor_search | v doctor_schedule | v availability search | v Present slots | v User selects slot | v Re-check availability | v Ask confirmation | YES | v appointment_create | v communication | v audit_log | v Confirmation

Example: User says “next Monday morning.” The agent finds Dr. Ahmed's working hours and available slots. User selects 11:30 AM. The agent asks for final confirmation. Only after confirmation does appointment_create run.

The create operation must re-check the slot immediately before writing. If another person took the slot, return a conflict and offer alternatives. Never claim success when the tool failed.

Example booking state

```
{ "intent": "book_appointment", "patient_id": "P1004", "doctor_id": "D102", "date": "2026-09-07", "time_range": "morning", "selected_slot": "11:30", "confirmation_required": true, "confirmed": false }
```

8. Rescheduling and Cancellation

RESCHEDULE find current appointment -> doctor schedule -> available slots -> user selection -> confirmation -> appointment_reschedule -> notification -> audit CANCEL find current appointment -> policy_rag -> check timing -> explain policy -> confirmation -> appointment_cancel -> notification -> audit

9. Multi-Tool Example — Missed Appointments

User: “Find patients who missed appointments this month, verify their insurance status, and prepare reminder messages for those whose insurance is still active.”

```
Planner | +--> appointment_search / analytics -> no-show list | +--> patient_search -> permitted records | +--> insurance_verify -> status | +--> filter active patients | +--> communication -> draft reminders | +--> human approval -> send if approved | +--> audit_log
```

10. Complex Operations Report

User: “Prepare this month's clinic operations report with appointment statistics, no-show rates, billing performance, insurance verification failures, and areas that need staff attention.”

```
Supervisor / Planner | +--> appointment metrics +--> no-show analytics +--> billing summary +--> insurance summary +--> policy_rag | v Insight synthesis | v report_generator -> PDF | v audit_log
```

Measured numbers should come from deterministic services. The LLM can explain trends, but the report should distinguish measured metrics from AI-generated observations.

11. Policy RAG and Web Search

Store administrative rules as versioned documents: cancellation, refund, appointment, working hours, communication, insurance and escalation policies.

```
Question -> policy_rag -> retrieve relevant chunks -> LLM interprets policy -> answer Example: "Can I cancel 12 hours before my appointment?" -> retrieve policy -> retrieve appointment timing -> compare -> explain rule
```

Web search is for permitted public/external information. For organization-specific rules, prefer internal policy RAG. Neither should be treated as an uncontrolled medical advice engine.

12. Human-in-the-Loop

Consequential actions can require explicit approval: booking, cancellation/rescheduling, bulk communication, report distribution and selected escalations. LangGraph can pause at an approval state and resume after approval.

```
Agent proposes action | v Approval state / \ Approve Reject | | v v Execute Stop safely | v Audit log
```

13. RBAC and Data Scope

The final backend supports PATIENT, RECEPTIONIST, BILLING/INSURANCE STAFF and ADMIN. Authorization also uses data scope: OWN_DATA, ASSIGNED_DATA, DEPARTMENT_DATA or ALL_DATA.

Capability	Patient	Receptionist	Billing/Insurance	Admin
Own profile/appointments	Yes	Yes	Yes	Yes
Other patients	No	Yes	Limited	Yes
Doctor schedules	Yes	Yes	Limited	Yes
Book appointment	Own	Yes	No	Yes
Cancel/reschedule	Own	Yes	No	Yes
Billing	Own/Limited	No	Yes	Yes
Insurance	Own/Limited	Limited	Yes	Yes
Analytics	No	Limited	Limited	Yes
Reports	No	Limited	Yes	Yes
Policy RAG	Yes	Yes	Yes	Yes
Web search	Yes	Yes	Yes	Yes
Communication	Own	Yes	Limited	Yes
Audit logs	No	No	Limited	Yes
Staff management	No	No	No	Yes

A patient can book/view their own appointment but cannot inspect another patient's records or organization-wide analytics. A receptionist can manage authorized appointments. Billing staff can access billing/insurance. Administrators can access organization-wide operations.

14. Authorization Examples

- Allowed:** P1004 asks “What appointments do I have tomorrow?” The agent requests appointment_search with OWN_DATA; backend verifies ownership and allows it.
- Denied:** P1004 asks “Show me P2008’s appointments.” The permission engine denies it before protected data is returned.
- Allowed:** A receptionist asks for P1004’s appointments; role and scope permit it.

Critical rule: The LLM never grants itself permission. It requests a capability; backend authorization makes the final decision.

15. Authentication and FastAPI Backend

LOGIN -> email/password -> verify credentials -> JWT -> user_id + role POST /api/chat -> validate JWT -> trusted user context -> LangGraph -> tool request -> permission engine -> service/tool -> result -> response

Suggested API endpoints

Endpoint	Purpose
POST /api/auth/login	Authenticate and issue JWT
POST /api/auth/refresh	Refresh token
GET /api/me	Current identity and role
POST /api/chat	Natural-language agent endpoint
GET /api/patients/{id}	Authorized patient data
GET /api/appointments	Authorized appointment search
POST /api/appointments	Authorized booking
PATCH /api/appointments/{id}	Authorized rescheduling
DELETE /api/appointments/{id}	Authorized cancellation
GET /api/doctors	Doctor search
GET /api/doctors/{id}/schedule	Doctor schedule
GET /api/reports/{id}	Authorized report retrieval
POST /api/approvals/{id}/approve	Approve pending action
POST /api/approvals/{id}/reject	Reject pending action
GET /api/audit-logs	Authorized audit access

16. Backend Folder Structure

```
healthcare-agent/
├── app/
│   ├── main.py
│   └── api/
│       ├── auth.py
│       ├── chat.py
│       ├── patients.py
│       ├── appointments.py
│       ├── doctors.py
│       ├── billing.py
│       ├── reports.py
│       ├── agents/
│       │   ├── graph.py
│       │   ├── state.py
│       │   ├── planner.py
│       │   ├── nodes/
│       │   │   ├── tools/
│       │   │   ├── patient.py
│       │   │   ├── appointments.py
│       │   │   ├── calendar.py
│       │   │   ├── billing.py
│       │   │   ├── insurance.py
│       │   │   ├── analytics.py
│       │   │   ├── search.py
│       │   │   ├── rag.py
│       │   │   ├── communication.py
│       │   │   ├── reports.py
│       │   │   ├── escalation.py
│       │   │   └── services/
│       │       ├── patient_service.py
│       │       ├── appointment_service.py
│       │       ├── billing_service.py
│       │       ├── insurance_service.py
│       │       ├── auth/
│       │       │   ├── jwt.py
│       │       │   ├── permissions.py
│       │       │   ├── dependencies.py
│       │       │   ├── models/
│       │       │   │   ├── user.py
│       │       │   │   ├── patient.py
│       │       │   │   ├── doctor.py
│       │       │   │   ├── appointment.py
│       │       │   │   ├── billing.py
│       │       │   │   ├── audit.py
│       │       │   │   ├── database/
│       │       │   │   ├── connection.py
│       │       │   │   ├── migrations/
│       │       │   │   ├── tests/
│       │       │   │   ├── agent/
│       │       │   │   ├── tools/
│       │       │   │   ├── services/
│       │       │   │   ├── auth/
│       │       │   │   └── api/
│       │       └── Dockerfile
│       └── docker-compose.yml
├── requirements.txt
└── .env
```

Separation of concerns: agent = reasoning/state; tools = typed capabilities; services = business logic; FastAPI = HTTP boundary; permission engine = authorization; models/database = persistence; audit = traceability.

17. Database Design

Table	Purpose / key fields
users	id, email, password_hash, role, status, timestamps
patients	id, user_id, name, administrative identifiers, contact
doctors	id, name, department, schedule/status
appointments	id, patient_id, doctor_id, start/end, status, created_by
appointment_events	appointment_id, event_type, actor, timestamp
billing_records	patient_id, invoice_id, amount, status, due_date
insurance_records	patient_id, provider, policy reference, status, expiry
reports	type, created_by, file reference, status, timestamps
approvals	requester, action, payload, status, approver
audit_logs	actor, role, tool, target, action, result, timestamp
policy_documents	title, version, source, metadata
conversation_threads	user_id, graph/checkpoint metadata, timestamps

18. Permission Engine

```
authorize(user, tool_name, scope, arguments) -> ALLOW / DENY PATIENT + appointment_search + OWN_DATA +  
patient_id=P1004 + authenticated user owns P1004 => ALLOW PATIENT + appointment_search + ALL_DATA => DENY
```

The permission layer checks both action and target data. This prevents privilege escalation through an otherwise legitimate tool.

19. Error Handling and Safety

Situation	Required behavior
Tool timeout	Retry when safe; otherwise report unavailable result.
Slot conflict	Return conflict and fetch alternatives.
Invalid identifier	Validate and ask for correction.
Unauthorized access	Deny without leaking protected data.
Unknown model tool	Reject and log validation failure.
External API failure	Safe fallback or clear unavailable response.
Ambiguous booking	Ask for missing doctor/date/time.
Clinical request	Do not diagnose; use appropriate escalation.
Bulk communication	Require authorization and approval.

20. Testing Strategy

Layer	Examples
Tool unit tests	Valid/invalid inputs and failures
Schema tests	Tool arguments validate correctly
Graph tests	Correct state transitions
Agent tests	Correct tool choice and sequences
Booking tests	Missing data, conflicts, confirmation and success
RBAC tests	Every role/scope combination
API tests	JWT, validation and status codes

Layer	Examples
Integration tests	Agent -> permission -> service -> PostgreSQL
Regression tests	Existing workflows remain correct
Evaluation	Tool-selection accuracy, unnecessary calls, completion/recovery

21. Complete End-to-End Booking Example

1. Patient logs in. 2. FastAPI validates JWT. 3. Backend loads user_id=P1004, role=PATIENT. 4. User sends: "Book me with Dr. Ahmed next Monday morning." 5. LangGraph classifies BOOK_APPOINTMENT. 6. doctor_search finds Dr. Ahmed. 7. doctor_schedule checks working hours. 8. Availability search finds open slots. 9. Agent presents slots. 10. Patient selects 11:30 AM. 11. Permission engine verifies own-account booking. 12. Agent asks for confirmation. 13. Patient confirms. 14. appointment_create re-checks the slot. 15. PostgreSQL stores the appointment. 16. communication prepares/sends confirmation. 17. audit_log records the action. 18. Agent returns confirmation. If step 14 reports a conflict: -> do not claim success -> fetch alternatives -> ask user to choose again.

This flow demonstrates authentication, stateful reasoning, multi-tool calls, authorization, persistence, communication, failure handling and auditing.

22. Complex Administrative Example

ADMIN | v "Generate this month's operations report." | v LangGraph Planner | +--> appointment metrics +--> no-show analytics +--> billing summary +--> insurance summary +--> policy RAG | v Insight synthesis | v report_generator -> PDF | v audit_log -> final response

23. Exact Building Roadmap

1. Finalize requirements and tool contracts: inputs, outputs, errors and side effects.
2. Create synthetic patients, doctors, schedules, appointments, billing and insurance data.
3. Implement all standalone tools.
4. Unit-test every tool.
5. Connect LangChain tools to Llama/Qwen through Groq.
6. Create LangGraph state and planner.
7. Implement dynamic tool selection and execution/observation loop.
8. Implement stateful appointment booking.
9. Implement rescheduling and cancellation.
10. Add analytics, billing and insurance workflows.
11. Add policy RAG and web search.
12. Add communication, reports and escalation.
13. Add retries, validation and safe failure handling.
14. Run full agent tests and evaluate tool selection.
15. Freeze a stable agent version.
16. Build FastAPI and PostgreSQL backend.
17. Implement JWT authentication and user/role models.
18. Implement RBAC and data-scope permissions.
19. Connect FastAPI to the existing LangGraph agent.
20. Add approvals and audit logging.
21. Run role-based security and integration tests.

22. Add Redis/background jobs where useful.

23. Dockerize the stack.

24. Deploy and run end-to-end tests.

24. Agent Test Scenarios

- “What appointments do I have tomorrow?” — authorized single-tool lookup.
- “Show Dr. Ahmed's schedule tomorrow.” — doctor search/schedule.
- “Book me with Dr. Ahmed next Monday morning.” — multi-turn booking.
- “Book the 11:30 slot.” — state continuation and confirmation.
- Selected slot becomes unavailable — conflict recovery.
- “Move my appointment to Wednesday.” — rescheduling.
- “Cancel my appointment.” — policy + appointment + confirmation.
- “Find patients who missed appointments this month.” — analytics.
- “Verify insurance for these patients.” — multi-tool workflow.
- “Generate the monthly operations report.” — analytics + report generation.
- Patient tries to access another patient's appointment — deny.
- Receptionist searches an authorized patient — allow.
- Billing staff accesses billing — allow; unrelated tools remain restricted.
- Admin generates organization-wide report — allow.
- Search provider fails — graceful failure.
- LLM requests an unknown tool — reject and audit.
- Communication action requires approval — pause and resume.
- Clinical/sensitive request — safe escalation rather than diagnosis.

25. Final Architecture Summary

```
CLIENT | v +-----+ | FastAPI | +-----+ | JWT / Identity | v +-----+ | LangGraph Agent | |
Supervisor/State | +-----+ | Planner/Reasoner | Tool Selection | Permission Engine / \ ALLOW DENY | |
v v Execute Safe reply | +-----+ | | v v v DB External RAG/Search services APIs \ | /
\_____|_____/ | Tool Results | Agent Reasoning | More tools? / \ Yes No | | +----> Response | Audit/Logs
```

Final design principle

This is a **permission-aware, stateful agentic workflow engine**. LangGraph handles orchestration; LangChain exposes typed tools; deterministic services perform operations; FastAPI provides the secure backend boundary; PostgreSQL persists data; RBAC controls access; RAG supplies administrative policies; and approval/audit layers make consequential actions traceable.

The architecture can later expand to a supervisor plus specialized scheduling, billing and administrative agents without changing the core design.